

Cliente REST Web Services - Seguridad con Jakarta EE



Cliente REST con Seguridad Básica (Jakarta EE)

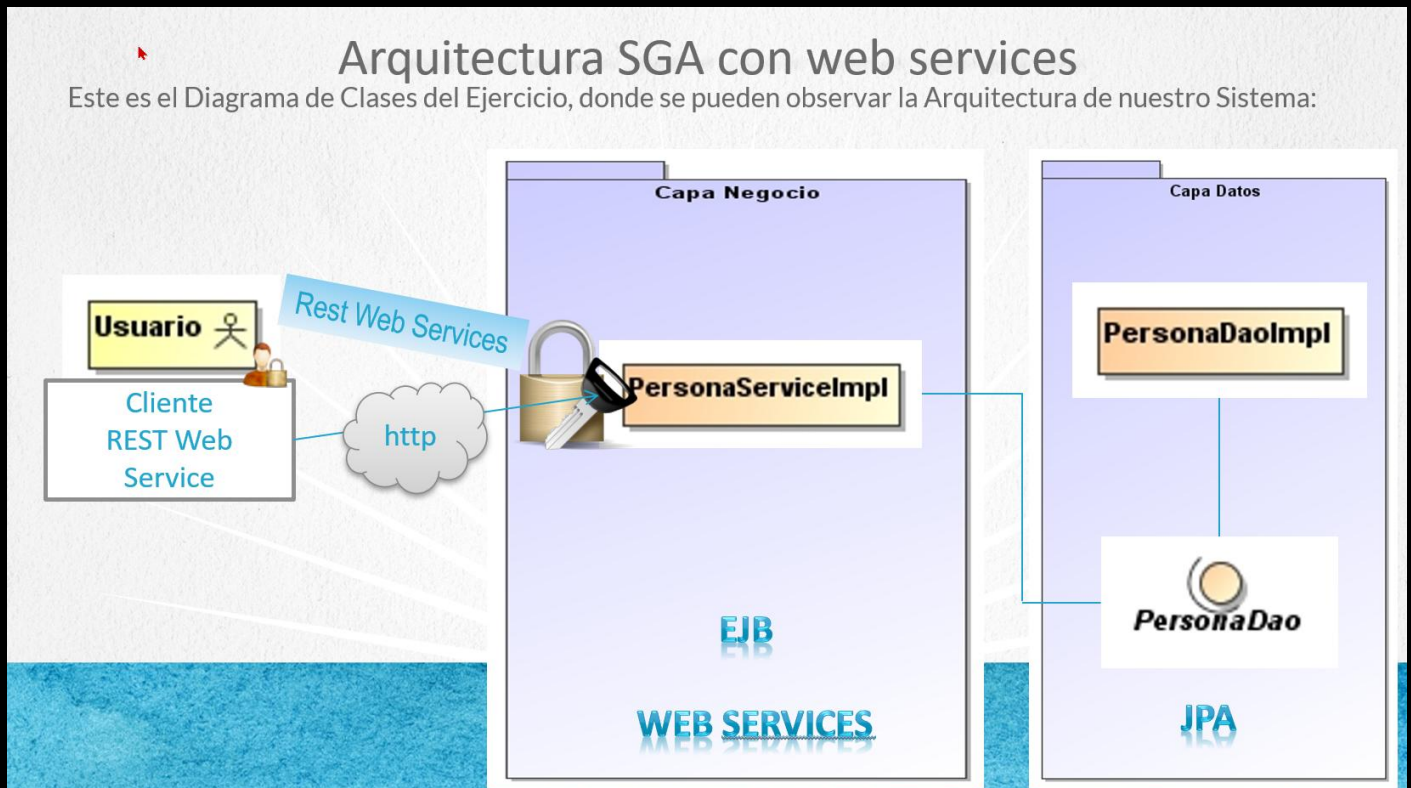
Introducción

En esta guía aprenderás a **modificar un cliente RESTful** para que pueda autenticar llamadas a un servicio web seguro utilizando **autenticación HTTP básica**.

 El servicio web del lado del servidor está protegido mediante restricciones de seguridad, por lo que necesitamos configurar correctamente el cliente para que se autentique al hacer peticiones.

 En esta lección trabajaremos con:

- Configuración de autenticación básica en un cliente REST.
- Uso de la clase `HttpAuthenticationFeature` para enviar usuario y contraseña.
- Configuración de `ClientConfig` y creación de instancias de cliente.
- Modificación de una clase de prueba existente para integrarle seguridad.



🔧 Paso 1: Crear clase `PruebaRestWS.java`

Ruta:

`ClienteSgaRestWs/src/main/java/test/PruebaRestWS.java`

Código:

```

package test;

import domain.Persona;
import jakarta.ws.rs.client.*;
import jakarta.ws.rs.core.MediaType;
import org.glassfish.jersey.client.ClientConfig;
import org.glassfish.jersey.client.authentication.HttpAuthenticationFeature;

public class PruebaRestWS {
    public static void main(String[] args) {

        HttpAuthenticationFeature feature =
            HttpAuthenticationFeature.basicBuilder()
  
```

```

        .nonPreemptive().credentials("admin", "admin").build();

ClientConfig clientConfig = new ClientConfig();
clientConfig.register(feature);

Client cliente = ClientBuilder.newClient(clientConfig);

WebTarget webTarget =
    cliente.target("http://localhost:8080/sga-jee-web/webservice").path("/personas");
//Proporcionamos un idPersona valido
Persona persona = webTarget.path("/2").request(MediaType.APPLICATION_XML).get(Persona.class);
System.out.println("Persona recuperada:" + persona);
    }
}

```

Explicación paso a paso:

1. Autenticación Básica

Se utiliza la clase `HttpAuthenticationFeature` para enviar las credenciales (admin, admin).

```

HttpAuthenticationFeature feature = HttpAuthenticationFeature.basicBuilder()
    .credentials("admin", "admin")
    .build();

```

2. Configuración del Cliente

Registramos la autenticación en un `ClientConfig`.

```

ClientConfig clientConfig = new ClientConfig();
clientConfig.register(feature);

```

3. Creación del Cliente

Se crea un cliente usando `ClientBuilder` con la configuración personalizada.

```

Client cliente = ClientBuilder.newClient(clientConfig);

```

4. Target al Servicio Web

Se define la URL del recurso y el path del método a invocar.

```

WebTarget webTarget = cliente.target("http://localhost:8080/sga-jee-
web/webservice")
    .path("/personas/1");

```

5. Petición GET

Se hace la llamada al servicio y se recupera una `Persona`.

```

Persona persona =
webTarget.request(MediaType.APPLICATION_XML).get(Persona.class);

```

```
System.out.println("Persona recuperada: " + persona);
```



Ejecución de la Clase



Paso para probar el cliente:

- Asegúrate de que el servidor GlassFish esté levantado y que el servicio `sga-jee-web` esté desplegado correctamente.
- Da clic derecho sobre `PruebaRestWS.java` y selecciona **Run** o **Run File**.



Si todo está correcto, deberías ver en la consola la persona recuperada con el ID proporcionado.



Paso 2: Modificar clase `TestPersonaServiceRS.java`

Agregamos la autenticación al archivo `TestPersonaServiceRS.java`



Ruta del archivo:

`ClienteSgaRestWs/src/main/java/test/TestPersonaServiceRS.java`

Código:

```
HttpAuthenticationFeature feature
= HttpAuthenticationFeature.basicBuilder()
    .nonPreemptive().credentials("admin", "admin").build();

ClientConfig clientConfig = new ClientConfig();
clientConfig.register(feature);

Client cliente = ClientBuilder.newClient(clientConfig);
```



Explicación:

- Se agregó **autenticación HTTP básica** de la misma forma que en la clase `PruebaRestWS.java`.
- Se reemplazó el cliente original por uno configurado con autenticación.
- Ahora ya se pueden ejecutar las operaciones de:
 - **Listar personas**
 - **Agregar persona**
 - **Modificar persona**
 - **Eliminar persona**

El uso del `ClientConfig` permite reutilizar la configuración de seguridad en todas las operaciones del cliente.

Código Final Completo

Archivo:

ClienteSgaRestWs/src/main/java/test/TestPersonaServiceRS.java

```
package test;

import domain.Persona;
import java.util.List;
import jakarta.ws.rs.client.*;
import jakarta.ws.rs.core.*;
import org.glassfish.jersey.client.ClientConfig;
import org.glassfish.jersey.client.authentication.HttpAuthenticationFeature;

public class TestPersonaServiceRS {

    //Variables que vamos a utilizar
    private static final String URL_BASE = "http://localhost:8080/sga-jee-web/webservice";
    private static WebTarget webTarget;
    private static Persona persona;
    private static List<Persona> personas;
    private static Invocation.Builder invocationBuilder;
    private static Response response;

    public static void main(String[] args) {
        HttpAuthenticationFeature feature
            = HttpAuthenticationFeature.basicBuilder()
                .nonPreemptive().credentials("admin", "admin").build();

        ClientConfig clientConfig = new ClientConfig();
        clientConfig.register(feature);

        Client cliente = ClientBuilder.newClient(clientConfig);

        //Leer una persona (metodo get)
        webTarget = cliente.target(URL_BASE).path("/personas");
        //Proporcionamos un idPersona valido
        persona = webTarget.path("/1").request(MediaType.APPLICATION_XML).get(Persona.class);
        System.out.println("Persona recuperada:" + persona);
    }
}
```



```
//Leer todas las personas (metodo get con readEntity de tipo List<>
response = webTarget.request(MediaType.APPLICATION_XML).get();
personas = response.readEntity(new GenericType<List<Persona>>() {
});
System.out.println("\nPersonas recuperadas");
personas.forEach(System.out::println);

//Agregar una persona (metodo post)
Persona nuevaPersona = new Persona();
nuevaPersona.setNombre("Carlos");
nuevaPersona.setApellido("Miranda");
nuevaPersona.setEmail("cmiranda3@mail.com");
nuevaPersona.setTelefono("99887700");

invocationBuilder = webTarget.request(MediaType.APPLICATION_XML);
response = invocationBuilder.post(Entity.entity(nuevaPersona, MediaType.APPLICATION_XML));
System.out.println("");
System.out.println(response.getStatus());
//Recuperamos la personas recién agregada para después modificarla y al final eliminarla
Persona personaRecuperada = response.readEntity(Persona.class);
System.out.println("Persona agregada:" + personaRecuperada);

//Modificar la persona (metodo put)
//persona recuperada anteriormente
Persona personaModificar = personaRecuperada;
personaModificar.setApellido("Ramirez");
String pathId = "/" + personaModificar.getIdPersona();
invocationBuilder = webTarget.path(pathId).request(MediaType.APPLICATION_XML);
response = invocationBuilder.put(Entity.entity(personaModificar, MediaType.APPLICATION_XML));

System.out.println("");
System.out.println("response:" + response.getStatus());
System.out.println("Persona modifica:" + response.readEntity(Persona.class));

//Eliminar una persona
//persona recuperada anteriormente
Persona personaEliminar = personaRecuperada;
String pathEliminarId = "/" + personaEliminar.getIdPersona();
invocationBuilder = webTarget.path(pathEliminarId).request(MediaType.APPLICATION_XML);
response = invocationBuilder.delete();
System.out.println("");
System.out.println("response:" + response.getStatus());
System.out.println("Persona Eliminada" + personaEliminar);
}
}
```

Ejecución de la clase `TestPersonaServiceRS`

Ejecutamos la clase `TestPersonaServiceRS` con la nueva configuración y debería pasar la autenticación y realizar correctamente todas las operaciones CRUD contra el servicio web seguro.

Conclusión actualizada

En este ejercicio:

-  Aprendimos a pasar autenticación básica desde el cliente REST.
-  Creamos una nueva clase de prueba `PruebaRestWS.java`.
-  Y modificamos la clase existente `TestPersonaServiceRS.java` para incluir la configuración de autenticación, sin necesidad de duplicar lógica ni clases.

Esto asegura que todo el cliente funcione de forma segura contra servicios protegidos.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)